
GPT-Style LLM From Scratch

Complete Developer & Learner Documentation

Arjun Bhattarai

Second-Year B.E. Computer / Information Technology Student

PURE PYTORCH • FIRST PRINCIPLES • ZERO BLACK-BOX APIS

This document serves as a comprehensive architectural and conceptual guide accompanying the source code repository. It explains the theory, design decisions, and pipeline flow behind each modular component, ensuring learners can grasp the micro-level engineering dynamics of modern language models without getting lost in implementation details.

1. Project Overview & Motivation

This project implements a complete GPT-style Generative Pre-trained Transformer entirely from scratch using PyTorch. The overarching objective is to demystify the inner mechanics of state-of-the-art language models by constructing every underlying operational framework—including custom tokenizers, multi-head causal attention tensors, multi-stage training configurations, downstream optimization, and automated evaluation architectures—from basic math and first principles.

While standard modern workflows interface with LLMs strictly through high-level abstract frameworks or closed APIs, this codebase strips away external wrappers. By implementing each layer manually, developers can trace exactly how text is mapped to high-dimensional spaces, processed natively in parallel, optimized against raw corpora, and systematically structured to follow complex human prompts.

Why Build From Scratch?

- **Tensor-Level Verification:** Gain structural intuition for keys, queries, values, and causal masks at the raw matrix level rather than high-level concept abstractions.
- **Deterministic Weight Mapping:** Validating custom model layers against official OpenAI GPT-2 checkpoints provides mathematical proof of architectural precision.
- **Practical Transfer Learning:** Fine-tuning a structured, non-trivial checkpoint over standard tasks highlights real-world adaptation workflows.
- **Closed-Loop Instruction Pipeline:** Designing Supervised Fine-Tuning (SFT) routines paired with isolated local judges gives a complete engineering look at the modern model lifecycle.

2. Prerequisites & Background Knowledge

To acquire the deepest value out of this architecture, a foundational understanding of the following technical domains is highly recommended:

Core Topic	Relevance to Project Integration
Python (Intermediate)	The entire codebase, data pipeline processing, and formatting utilities are engineered natively in Python.
PyTorch Foundations	Deep familiarity with raw matrix structures (Tensors), automatic differentiation (Autograd), <code>nn.Module</code> subclassing, and basic optimizer routines.
Linear Algebra	High-dimensional matrix multiplications form the fundamental backend math governing all attention transformations.
Probability & Statistics	Categorical token distributions, Softmax normalization boundaries, and stochastic decoding/sampling behaviors.
Deep Learning Basics	Standard forward passes, backward error propagation, regularized loss optimization, and dense layer mechanics.
Text Tokenization	Conceptual understanding of processing unstructured strings into numerical, indexed vector formats.

Recommended Core Literature: For foundational theory, see *"Attention Is All You Need"* (Vaswani et al., 2017) alongside the original OpenAI generative pretraining publications. The engineering maps built inside this framework are designed to systematically materialize those core concepts step-by-step.

3. Tokenization & Preprocessing

Neural architectures cannot compute characters or raw strings directly; text data must first be broken down and vectorized into distinct mathematical representations.

3.1 What is a Token?

A token represents the atomic unit of text parsing ingested by the network. Depending on configuration limits, a token can represent a complete word, a character, or a subword fragment. This project builds a custom regex-based word tokenizer as an introductory learning module, and interfaces directly with OpenAI's BPE-based `tiktoken` library for production-grade GPT-2 model replication.

3.2 Vocabulary Construction

A vocabulary defines a closed, immutable mapping between unique structural text tokens and unique integer coordinates. The model is strictly constrained to processing indices defined inside this dictionary. A designated unknown or special out-of-vocabulary (OOV) token handles arbitrary structural fragments parsed at inference time that were absent during training phases.

3.3 Encoding & Decoding

Encoding acts as the processing gate, parsing raw, unformatted text into continuous blocks of numerical token IDs. Conversely, decoding maps multi-dimensional integer outputs out of the model network back into readable human strings. Both pipelines are strictly essential: encoding formats data for forward ingestion, while decoding reads out generative inferences.

3.4 Sliding Window Dataset

To facilitate foundational next-token autoregressive pretraining, raw source documents are split into overlapping chunks bounded by a fixed context size. For any given slice, the input tensor sequence spans indices $0 \rightarrow N-1$, while the paired target shift tensor spans positions $1 \rightarrow N$. This structural offset explicitly trains the model to map historical textual sequences to immediate target lookaheads.

3.5 Embeddings

Raw token integers possess no intrinsic geometric proximity or semantic relational meaning. Embedding matrices translate discrete token integers into dense, continuous vector spaces initialized with spatial gradients. Because standard Transformer structures process all token blocks concurrently with no native sense of sequence order, positional embeddings are combined with token representations to explicitly introduce sequence tracking.

- **Token Embedding Table:** Maps discrete token IDs to dense vectors. Spatial Dimension: $[\text{vocab_size} \times \text{embed_dim}]$.
- **Positional Embedding Table:** Maps absolute sequential indices to directional vectors. Spatial Dimension: $[\text{context_length} \times \text{embed_dim}]$.

Both matrix outputs are summed together element-wise before being passed to the core Transformer block layers.

4. Transformer Architecture

The Transformer represents the premier architectural foundation enabling modern generative AI systems. By dropping recursive looping frameworks, it executes complete multi-token sequence calculations concurrently via attention layers.

4.1 Self-Attention Mechanics

Self-attention grants the network the power to compute relational associations across all token states in a sequence dynamically. Every input block projects three distinct continuous vectors: a Query (**Q**), a Key (**K**), and a Value (**V**).

Raw attention weights are computed by taking the dot product of the Query matrix with the transposed Key matrix, scaled down by the square root of the head dimensions to insulate gradients against flattening. Running these normalized vectors through a Softmax function generates an accurate probability distribution, which is then multiplied by the Value matrix to extract a contextually weighted representation.

4.2 Causal (Masked) Attention

During generative pretraining, a directional model must be prevented from looking ahead at future target phrases. Causal attention introduces a structural upper-triangular mask that overrides future token attention weights to negative infinity (**-inf**) right before the Softmax step. After applying Softmax, these forward positions are crushed to zero weight, ensuring historical contexts can never glean downstream target tokens.

4.3 Multi-Head Attention

Rather than relying on a single large attention step, Multi-Head Attention segments the feature dimension to run multiple distinct attention projections in parallel. Each head acts as an independent semantic subspace, allowing the model to simultaneously track syntactic formatting, relational links, and contextual dependencies across separate channels. The independent head states are concatenated together and linearly projected back to match the foundational model dimensions.

4.4 Feed-Forward Network (FFN)

Following attention merging, the representation of each token independently passes through an isolated fully-connected feed-forward block. This module consists of two distinct linear layers wrapping a non-linear Gaussian Error Linear Unit activation function. Structurally, the internal hidden layer dimension scales to 4x the primary model embedding dimension to facilitate non-linear token transformations.

4.5 GELU Activation

Gaussian Error Linear Units (GELU) provide smooth non-linear gating functions within the FFN layers. Unlike traditional ReLU functions that abruptly clip negative coordinates to absolute zero, GELU smoothly scales gradients based on a Gaussian cumulative distribution function. This continuous curve enhances backpropagation performance and serves as the modern standard for autoregressive systems.

4.6 Layer Normalization

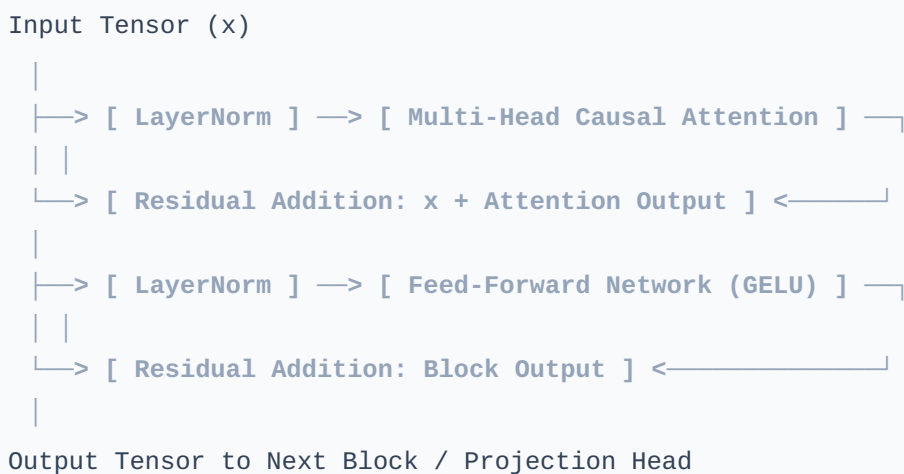
Layer Normalization enforces structural stability throughout training by standardizing activation scales across the feature dimensions of individual tokens. In contrast to Batch Normalization, LayerNorm

calculates metrics independently for each token sequence, optimizing it for variable-length processing blocks. This architecture uses a modern 'Pre-LN' approach, applying normalization directly before the self-attention and FFN sub-layers.

4.7 Residual Connections

Deep architectures are frequently bottlenecked by vanishing optimization signals. To fix this, every primary block utilizes residual skip connections where the raw layer input is added directly back to the sub-layer output. This provides clean, uninterrupted highways for gradients to flow backward through deep stacks without undergoing severe attenuation.

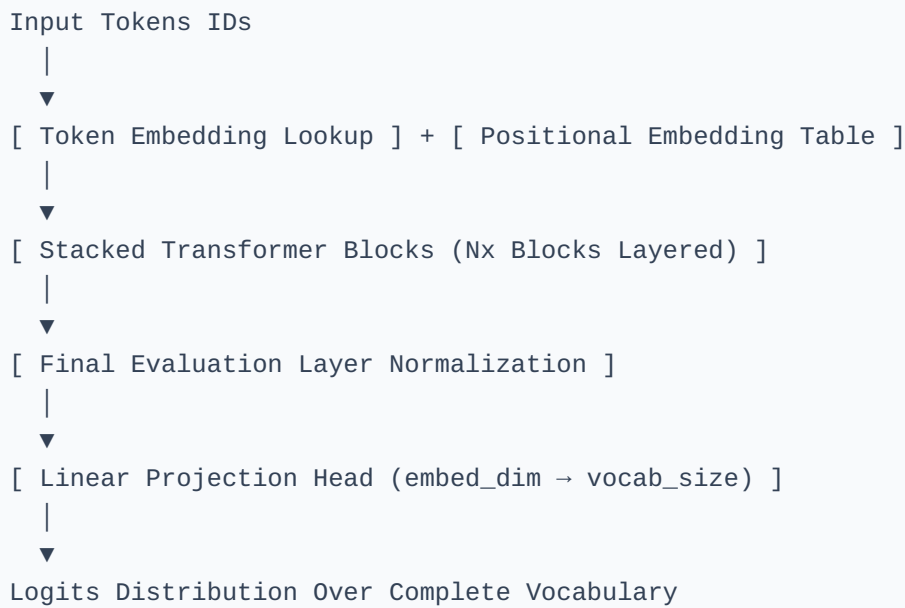
4.8 Full Transformer Block Pipeline



5. GPT-Style Model Design

The GPT paradigm functions strictly as a decoder-only Transformer configuration. By excluding the cross-attention encoder structures seen in standard machine translation, it remains purely autoregressive—generating text one token at a time based entirely on past contexts.

5.1 Architecture Mapping



5.2 Hyperparameters Reference Table

Hyperparameter Ticker	Functional Structural Description	GPT-2 Baseline Value
vocab_size	Total count of unique tokens tracked inside the vocabulary table.	50,257
context_length	Maximum structural token sequence limit per sequence block.	1,024
embed_dim	Feature length sizing of dense continuous token vectors.	768
num_heads	Parallel calculation channels configured per block layer.	12
num_layers	Total count of stacked Transformer blocks forming the model depth.	12
ffn_hidden_dim	Internal width multiplier of the feed-forward layer (4x embed).	3,072
dropout	Regularization frequency applied to suppress neural overfitting.	0.1

5.3 Output Layer: Logits

The terminal layer projects token features from `embed_dim` down to `vocab_size`, producing raw unnormalized prediction scores called logits. These scores represent the raw structural confidence metrics computed for potential next-token selections before being converted into valid probabilities via Softmax execution.

6. Pretraining

Pretraining constitutes the foundational phase where raw network parameters learn general grammar, syntax, and world relationships using self-supervised datasets without manual human labels.

6.1 Objective: Next-Token Prediction

For every token index in the batch sequence, the network optimizes to predict the matching down-sequence index. Given a training array $[t_1, t_2, t_3, t_4]$, the model resolves t_2 from t_1 , t_3 from the history $[t_1, t_2]$, and t_4 from $[t_1, t_2, t_3]$. Thanks to causal masks, these calculations occur concurrently for all tokens in a single forward pass.

6.2 Loss Function: Cross-Entropy

Calculations evaluate the model's categorical distribution against a target one-hot vector representation of the true sequence index. Cross-entropy loss penalties scale based on the distance between the predicted probability and the true target index. Minimizing this objective function pushes the model to assign high probability mass to correct token targets.

6.3 Optimizer: AdamW

AdamW serves as the standard optimization routine for training robust Transformers. It decouples weight decay calculations from standard gradient updates, ensuring reliable scaling over long training runs. By tracking the first and second moments of gradients for each parameter independently, it provides stable optimization updates.

6.4 Evaluation Metric: Perplexity

Perplexity (PPL) serves as the primary metric for language model quality, calculated directly as the exponent of the average cross-entropy loss:

$$PPL = \exp(\text{loss})$$

A perplexity rating of N indicates the model is as uncertain as choosing uniformly from N equally likely tokens at that step. Lower scores confirm a more confident model.

6.5 Train / Validation Structural Split

The corpus is split into distinct training datasets used to update model parameters and validation sets held out to monitor generalization. Tracking both training and validation tracks real-world convergence: if training loss collapses while validation loss jumps, the model is overfitting to the training set.

7. Text Generation Strategies

During inference, text generation runs autoregressively: the model generates a single token, appends it to the context, and feeds the updated sequence back in to project the next token. Sampling configurations heavily dictate overall generation quality and diversity.

7.1 Greedy Decoding

This strategy selects the token with the absolute highest probability at every step. While deterministic and useful for strict formatting tasks, greedy decoding frequently gets stuck in repetitive loops and produces rigid, unnatural text patterns.

7.2 Temperature Scaling

Temperature serves as a hyperparameter that scales raw logits before the Softmax normalization step. Dividing the logit array by a temperature value (T) changes the shape of the resulting probability curve:

- $T < 1$: Sharpens the distribution curve, making choices highly conservative and confident.
- $T > 1$: Flattens the distribution curve, raising the probability of creative or unexpected tokens.
- $T = 1$: Leaves the original computed Softmax probabilities unchanged.

7.3 Top-k Sampling

This approach limits the sampling pool strictly to the k highest-scoring tokens in the vocabulary. Lower-ranked tokens have their logits overwritten to negative infinity ($-\infty$) before Softmax calculation. This effectively eliminates rare, nonsensical token choices while maintaining healthy variation among top candidates.

7.4 Multinomial Sampling

Once logits are adjusted for temperature and filtered through top-k limits, they are passed through a final Softmax step to build a clean probability distribution. The next token is then selected via multinomial distribution drawing. While high-probability tokens remain the most likely choices, lower-ranked candidates retain a small chance of selection, leading to more natural text generation.

7.5 End-of-Sequence (EOS) Handling

A designated end-of-sequence (EOS) token explicitly marks text completion. When generated, the processing loop halts immediately. Without proper EOS rules, inference runs would continue generating filler text until hitting the absolute sequence length limit.

8. GPT-2 Weight Loading

Training large models from scratch requires massive compute resources. To bypass this bottleneck, this architecture includes a weight-loading utility that pulls public, pretrained OpenAI weights directly into our custom code structures.

8.1 OpenAI GPT-2 Parameter Formats

Model Identifier	Parameter Count	Layer Count	Attention Heads	Embedding Size
GPT-2 Small	124M	12	12	768
GPT-2 Medium	355M	24	16	1024
GPT-2 Large	774M	36	20	1280
GPT-2 XL	1.5B	48	25	1600

8.2 Weight Mapping and Transposition

OpenAI's checkpoint structures use custom naming standards and tensor layouts that differ from this custom implementation. The mapping utility translates each checkpoint key to its exact match in our native architecture. For example, OpenAI stores attention projections as fused, combined QKV blocks; our loader explicitly splits these matrices apart before assigning them. It also runs shape validation checks on all incoming arrays to catch dimension mismatches early.

8.3 Transferred Operational Parameters

- **wte & wpe:** Token and positional embedding tables mapping inputs to continuous vectors.
- **QKV Projections:** Core attention projection matrices and bias variables.
- **Output Projections:** Linear projection parameters that merge attention head outputs.
- **FFN Parameters:** Dual layered linear matrix weights configured across every block.
- **LayerNorm Overhauls:** All target gain and bias parameters distributed per layer.
- **Head Alignment:** The final output projection layer (tied to token embeddings per GPT-2 design).

9. Checkpointing

Because training deep architectures demands significant time and hardware compute, saving intermediate progress is essential. Checkpointing serializes the entire training state to disk, allowing runs to resume cleanly after interruptions and preserving top-performing models.

Crucial State Persistence Requirement: Resuming a run without reloading optimizer parameters can cause severe training instability. The AdamW optimizer tracks momentum histories across training loops; resetting these metrics to zero forces an unstable re-initialization phase. Saving the entire state ensures seamless training continuity.

Saved State Components:

- `model_state_dict`: Every active learned weight tensor across all layers.
- `optimizer_state_dict`: First and second moment gradient records tracked by AdamW.
- `Tracking Progress`: Integer trackers recording the current training epoch and step count.
- `Historical Diagnostics`: Complete lists of training and validation loss metrics across runs.

10. SMS Spam Classification Fine-Tuning

Fine-tuning adapts a pretrained model to a specific downstream task using a much smaller labeled dataset. In this module, the base model is updated to handle binary text classification: identifying if an SMS message is spam (1) or ham (0).

10.1 Downstream Specialization Dynamics

Pretrained foundational models already possess a strong understanding of language structure, syntax, and context. Fine-tuning leverages this existing knowledge and adapts it to classification tasks far faster and with significantly less data than training a new model from scratch.

10.2 Dataset Preparation

The system uses the UCI SMS Spam Collection dataset, containing 5,574 English text messages labeled as spam or ham. This data is parsed into a Pandas DataFrame, converting labels to binary format (ham → 0, spam → 1).

10.3 Class Balancing Via Downsampling

The raw dataset is highly imbalanced, with legitimate ham messages significantly outnumbering spam entries. Training on imbalanced distributions biases the model toward majority classifications. To resolve

this, the ham class is randomly downsampled to match the spam count, ensuring equal representation during training.

10.4 Dataset Splits

The balanced data is shuffled and split into three sets: a training set to update model parameters, a validation set to monitor for overfitting, and a test set to provide an unbiased final evaluation.

10.5 Custom SpamDataset Wrapper

A custom PyTorch Dataset class processes incoming text sequences. It converts strings into token IDs using the tokenizer, truncates segments that exceed maximum length limits, and applies padding tokens to shorter sequences to maintain uniform batch shapes.

10.6 Classification Head Integration

While base models project logits across the entire vocabulary, classification tasks require a categorical choice. The system extracts the final token representation from the sequence, which contains contextual information from the entire message. A linear layer maps this vector from `embed_dim` to a 2-dimensional classification space optimized via cross-entropy loss.

10.7 Training Performance Diagnostics

Training Epoch	Training Accuracy Status	Validation Accuracy Status
Epoch 1	70.0%	72.5%
Epoch 2	82.5%	85.0%
Epoch 3	90.0%	90.0%
Epoch 4	100.0%	97.5%
Epoch 5	100.0%	97.5%

Total Execution Window: 15.82 minutes. The model achieved a 97.5% validation accuracy, demonstrating that pretrained language features transfer effectively to downstream tasks with minimal data overhead.

11. Instruction Fine-Tuning (SFT)

Supervised Fine-Tuning (SFT) trains a model to understand and follow human prompt instructions. This transition converts a raw next-token predictor into a functional, conversational assistant.

11.1 The Mechanics of SFT

During SFT, the model trains on curated datasets consisting of paired instructions and responses. It processes a formatted prompt and learns to generate the appropriate answer. While the model uses the same next-token prediction loss as pretraining, the training focus shifts to high-quality instruction-response structures.

11.2 Target Dataset Structure

The framework utilizes `instruction-data.json`, containing 1,100 structured entries from the `rasbt/LLMs-from-scratch` repository. Each entry contains three primary keys: an instruction defining the task, an optional input block providing context, and an output string showing the ideal response.

11.3 Data Configuration Breakdowns

- **Training Set (85%):** Used to calculate forward paths and update active weights.
- **Validation Set (5%):** Used to monitor convergence and tune hyperparameters.
- **Test Set (10%):** Held back for final evaluation checks.

11.4 Unified Prompt Formatting Template

Every entry passes through a standardized layout before tokenization, training the model to recognize instruction boundaries and response targets reliably:

```
Below is an instruction that describes a task. Write a response that appropriately completes the request.
```

```
### Instruction:
```

```
[Instruction text content inserted here]
```

```
### Input:
```

```
[Optional contextual input text or left empty]
```

```
### Response:
```

```
[Target response payload generated here]
```

11.4 Custom Processing via InstructionDataset

A specialized `InstructionDataset` handles tokenization using the `tiktoken` BPE framework. It parses the entire instruction-response string together as a single continuous sequence of integer IDs.

11.5 Custom Collate Functions and Batching

Because different prompt entries vary in length, they cannot be stacked into uniform batches automatically. A custom collate function standardizes shapes across sequences:

- Appends the explicit `<|endoftext|>` token to signal sequence completion.
- Applies padding tokens to match the length of the longest sequence in the batch.
- Constructs input tensors by grabbing indices `tokens[: -1]`.
- Constructs target tensors by slicing indices `tokens[1:]` to create the next-token shift.
- Overrides all padding positions in target tensors to a value of `-100`.

11.6 The Significance of `ignore_index=-100`

PyTorch's cross-entropy loss function includes a built-in `ignore_index` parameter. When the loss function encounters a target position set to `-100`, it zeroes out the error score for that index and excludes it from gradient calculations. This step is critical for handling batches of varying lengths: it ensures the model is never penalized for predictions on meaningless padding filler, preserving the purity of the training signal.

12. Training & Evaluation Pipeline

With data preparation complete, a supervised training loop manages the full fine-tuning process, emphasizing reproducibility and detailed metrics tracking.

12.1 Execution Steps of the Supervised Loop

- **Deterministic Seeding:** Sets global random seeds before any operations to ensure identical runs.
- **Device Allocation:** Automatically moves model parameters onto target hardware (CPU or GPU) before execution.
- **Gradient Reset:** Clears optimizer gradients at the start of each step to prevent gradient accumulation.
- **Loss Computation:** Runs forward passes to compute logits, then evaluates loss against shifted targets.
- **Error Propagation:** Executes backward passes to compute gradients, and updates model weights via AdamW.
- **Validation Checks:** Performs periodic evaluations on validation data to track generalization.

- **Token Accounting:** Tracks the total count of processed tokens to build clear data-efficiency curves.
- **Performance Benchmarking:** Measures wall-clock time from start to finish to evaluate training speed.

12.2 Training and Validation Loss Visualization

Loss metrics are plotted against both active training epochs and the total number of tokens processed. Tracking loss against token count provides a more accurate view of training efficiency across different runs, especially when changing batch sizes or evaluation frequencies.

12.3 Generating and Comparing Test Responses

After fine-tuning, the model generates outputs for every instruction in the test dataset. The system pairs these generated strings directly with the reference answers, creating a structured comparison for qualitative reviews and automated scoring pipelines.

12.4 Sanitized Checkpoint Management

The system saves the finalized weights using a sanitized filename derived from the model's configuration parameters. This checkpoint allows users to reload the model instantly for inference or downstream work without repeating the training process.

13. Automated Response Evaluation with Ollama

Evaluating generative model quality at scale can be difficult. This framework builds an automated evaluation pipeline using a local Ollama model to score test outputs quantitatively.

13.1 LLM-as-a-Judge Design Pattern

Ollama runs open-source language models locally via a straightforward REST API. In this project, it acts as an automated judge: it reviews the prompt instruction, the reference answer, and the model's generated response, then outputs a quality score. This approach is known as the "LLM-as-a-Judge" evaluation pattern.

13.2 Environment Verification via psutil

Before launching evaluations, the system uses the `psutil` library to verify that the Ollama server process is active. If the server is offline, the pipeline throws a clear error immediately instead of failing silently in the middle of a evaluation run, ensuring a stable environment before making API calls.

13.3 Local REST API Integration

The system's `query_model` function handles communication with the Ollama server using Python's standard `urllib.request` library, avoiding external HTTP dependencies. The integration applies strict configuration rules:

- **Temperature = 0:** Enforces deterministic behavior to guarantee identical scores for identical inputs across runs.
- **Fixed Operational Seed:** Ensures scoring criteria remain consistent across different evaluation sessions.
- **Raw Parsing Targets:** Returns full text responses for downstream formatting and extraction blocks.

13.4 Integer Scoring and Robust Error Handling

The evaluator model is instructed to output an integer score within a strict scale (e.g., 1-100). The pipeline parses this response to extract the numeric grade. If the judge outputs unexpected formatting, error handlers catch the parsing failure gracefully and skip the entry or apply a default score to prevent the evaluation run from crashing.

13.5 Batch Score Generation

The `generate_model_scores` function manages the evaluation loop across the test set. It sends entries to the judge, processes the numeric scores, and compiles the results. The final output is an array of numeric scores that can be averaged or analyzed to evaluate the model's instruction-following capabilities.

13.6 End-to-End Evaluation Pipeline Flow

```
[ Test Dataset Entries Loaded ]  
|  
▼  
[ Process Status Verification via psutil ]  
|  
▼  
[ Prompt Assembly (Instruction + Reference Answer + Model Response) ]  
|  
▼  
[ query_model API Delivery via urllib POST Request ]  
|  
▼  
[ Integer Score Extraction & Parsing Exception Guards ]  
|  
▼  
[ generate_model_scores Batch Aggregation ]  
|  
▼  
[ Final Compiled Metric Array and Quality Summary Report ]
```

14. End-to-End Pipeline Summary

The architecture flows through a series of connected steps, transforming raw text inputs into evaluated, instruction-following outputs:

1. Raw Text Document Ingestion
▼
2. Tokenization Splits (Custom Regex / Production BPE via tiktoken)
▼
3. Vocabulary Mapping & Sequence Integer Encoding
▼
4. Vector Conversion (Summed Token & Positional Embeddings)
▼
5. Transformer Stacks (Causal Self-Attention + FFN + Pre-LN + Residual Skip Layers)
▼
6. Self-Supervised Next-Token Prediction Pretraining Loop
▼
7. OpenAI GPT-2 Parameter Extraction and Weight Loading Mapping Utilities
▼
8. Checkpoint Serialization (Model Weights, Optimizer Moments, and Step Metrics)
▼
9. Autoregressive Inference Generation (Controlled via Temperature, Top-k, and Multinomial Sampling)
▼
10. SMS Classification Fine-Tuning Task Adaptation (Achieving 97.5% Accuracy Summary)
▼
11. Supervised Instruction Fine-Tuning (Custom InstructionDataset and Data Collator Implementations)
▼
12. Supervised Fine-Tuning Execution Loop (Loss Tracking and Progress Plotting Visualization)
▼
13. Test Dataset Inference Response Generation Profiles
▼
14. Automated Response Quality Grading (Ollama Local LLM-as-a-Judge Evaluation Pipeline)

15. Key Concepts Glossary

Core Architecture Term	First-Principles Technical Definition
Token	The atomic textual unit handled by the neural network (e.g., character, subword, or full word).
Tokenizer	The processing framework that converts raw text strings into numerical IDs and handles translation back to text.
Embedding	Continuous high-dimensional vectors representing tokens and positions, learned during optimization.
Logits	Raw, unnormalized output values generated by the model's projection layers before processing by Softmax.
Softmax	An activation function that normalizes an array of arbitrary values into a probability distribution that sums to 1.
Cross-Entropy Loss	An optimization metric that measures the distance between the model's predicted probability distribution and the true target values.
Perplexity	Calculated as $\exp(\text{loss})$; an intuitive metric showing language model confidence. Lower values indicate clearer generation.
Self-Attention	An architectural mechanism that allows individual tokens to dynamically weight and associate with all other positions across a sequence.
Causal Mask	A lower-triangular matrix configuration that blocks visibility into future positions, enforcing autoregressive directionality during training.
Multi-Head Attention	Splits attention layers into parallel processing subspaces to capture diverse semantic and syntactic relationships concurrently.
Query / Key / Value	Linear vector projections for attention routing: Query maps search targets, Key indicates content properties, Value holds the underlying information.
Residual Connection	A skip connection that adds a layer's input directly back to its output, preventing vanishing gradients across deep model structures.
Layer Normalization	Standardizes activation scaling across the feature dimension of independent tokens, promoting structural stability during updates.
GELU	Gaussian Error Linear Unit; a smooth non-linear activation function that optimizes gradient propagation relative to standard ReLU steps.

Core Architecture Term	First-Principles Technical Definition
AdamW	A variation of the Adam optimization routine that uncouples weight decay from gradient tracking; the industry standard for Transformer architectures.
Fine-Tuning	The process of updating a pretrained model's weights to adapt it to a specific, specialized task using smaller labeled datasets.
SFT	Supervised Fine-Tuning; training a model directly on formatted instruction-response pairs to develop assistant behaviors.
ignore_index=-100	A configuration parameter instructing PyTorch cross-entropy functions to ignore specified indices, shielding padding tokens from loss updates.
Collate Function	A custom data-handling utility that manages batching and applies uniform padding across inputs of varying lengths.
Temperature	A hyperparameter that scales logits before Softmax to manage generation variety: lower values reduce randomness, higher values increase output diversity.
Top-k Sampling	A decoding configuration that limits token generation choices to the top k candidates, removing unexpected or lower-ranked options.
Ollama	A local execution framework for running open-source models via REST APIs, used in this project to manage automated judge routing.
LLM-as-Judge	An evaluation paradigm where an independent, capable language model is utilized to grade text generated by other models.
Checkpoint	A serialized snapshot of model parameters and optimizer states saved to disk, enabling developers to resume training pipelines cleanly.
psutil	A cross-platform process and system monitoring library used here to check the active server status of local evaluation judges.
BPE	Byte-Pair Encoding; a subword tokenization algorithm that builds clean vocabularies by merging frequent character groups, utilized by tiktoken.
Transfer Learning	The machine learning technique of taking features learned from massive initial pretraining and applying them directly to smaller, specialized tasks.
Decoder-Only	

Core Architecture Term	First-Principles Technical Definition
	A Transformer variation that uses only the decoder stack with causal masking, optimized for generation tasks (e.g., the GPT family).

116. About the Author & Scope

This project was designed, architected, and built entirely from scratch by **Arjun Bhattarai**, a second-year B.E. Computer / Information Technology student, as a foundational deep learning portfolio project.

Every operational module—from raw regex tokenizers and multi-head causal attention tensors to the GPT-2 checkpoint weight loaders, fine-tuning structures, and local Ollama evaluation integrations—was engineered independently in PyTorch without relying on high-level orchestration abstractions like Hugging Face Transformers. The project focus shifts past shallow API calls to build a deep, firsthand engineering understanding of how modern language models are constructed, optimized, and evaluated.

Completed Framework Implementation Scope:

- Full autoregressive GPT-style Transformer engine written entirely from first principles.
- Custom weight-loading translation utility supporting all four official OpenAI GPT-2 size formats.
- Comprehensive pretraining infrastructure featuring real-time loss tracking and visualization updates.
- Downstream SMS classification fine-tuning pipeline, reaching a 97.5% validation accuracy.
- Supervised Fine-Tuning (SFT) architecture supported by custom Dataset utilities and collate formatting functions.
- Automated evaluation framework utilizing a local Ollama LLM-as-a-Judge pipeline.

"Built from scratch. Every line written with intent." — Arjun Bhattarai